

Analysing the Impact of Hidden Layers on the Performance of a Multilayer Perceptron (MLP)

Name: Narsing Lokesh

Student Id: 24129854

Github Link: <https://github.com/NarsingLokesh/mlp-hidden-layers-analysis.git>

1.Introduction

Machine learning models are widely used in many real-world applications such as banking, marketing, and healthcare. A neural network is one of the important types of models, which is capable of learning complex patterns from data. Multilayer Perceptron (MLP) is one of the most widely used neural networks. An MLP consists of three main parts: an input layer, one or more hidden layers, and an output layer. The hidden layers are responsible for transforming the input data into useful representations so that the model can make accurate predictions. However, selecting the correct number of hidden layers is an important design decision.

If a model contains too few hidden layers, it may not learn enough from the data, leading to underfitting. On the other hand, if it has too many hidden layers, it may become too complex and perform poorly on unseen data (overfitting). Hence, it is important to understand how the number of hidden layers affects model performance. The goal of this assignment is to analyse the impact of hidden layers on the performance of a Multilayer Perceptron using a real-world dataset. Different models with varying numbers of hidden layers are trained and compared using accuracy and loss metrics.

2.Technique Used - Multilayer Perceptron (MLP)

The main technique used in this assignment is the Multilayer Perceptron (MLP), it is also called “multilayer” because it contains multiple layers of neurons between the input and output. In each neuron the network receives inputs, applies weights, adds a bias, and passes the result through an activation function.

Implementation of MLP:

- The hidden layers use the ReLU (Rectified Linear Unit) activation function, which helps the model learn non-linear relationships efficiently.
- The output layer uses the Sigmoid activation function, which is suitable for binary classification problem.
- The model uses binary cross-entropy as the loss function because the target variable has two classes (yes or no).
- The Adam optimizer is used to update the model weights during training.

Dataset Description

The dataset used in this assignment is Banking Marketing Dataset, which has information about customers of a bank. The aim of the dataset is to find whether a customer will subscribe to a term deposit.

The dataset contains several features like:

- Age of the customer
- Job type
- Marital status
- Education level
- Account balance
- Loan status
- Contact details
- Previous campaign results

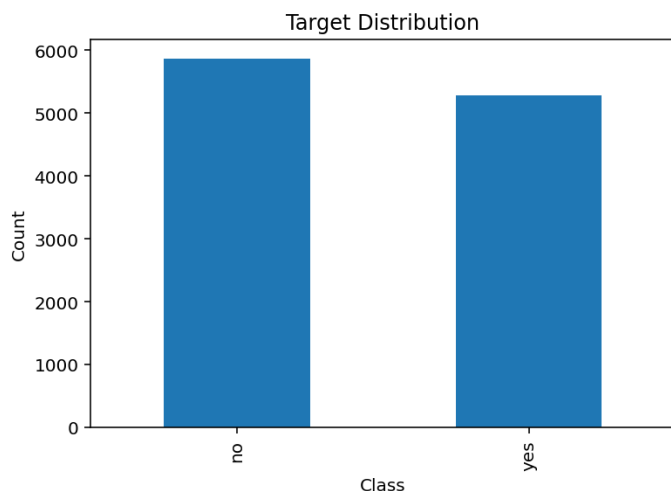
The target feature in the dataset is **deposit** – (Indicates whether the customer subscribed to a term deposit (yes or no)). This is a binary classification problem where the model predicts either 0 (no) or 1 (yes).

3.Exploratory Data Analysis (EDA)

Before building the model, exploratory data analysis was performed to understand the dataset.

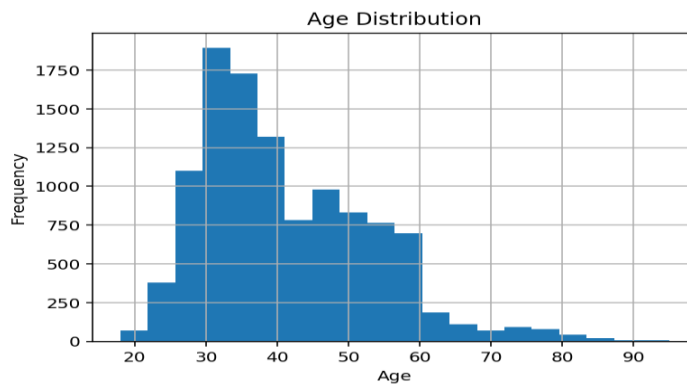
3.1 Target Distribution

- The first graph shows the distribution of the target variable.
- This graph displays how many customers subscribed to the deposit and how many did not. Understanding this distribution is important because it helps identify whether the dataset is balanced.
- A balanced dataset means that both classes are represented in similar proportions, which helps the model learn effectively. If the dataset is highly imbalanced, the model may become biased towards one class.
- In this case, both classes are present in sufficient numbers, allowing the model to learn patterns from both outcomes.



3.2 Age Distribution

- The second graph shows the distribution of customer ages.
- This graph helps in understanding how the age values are spread across the dataset. Most customers fall within a certain age range, indicating that the dataset represents a specific demographic group.
- Understanding feature distributions helps in interpreting model behaviour and ensures that the data is suitable for training



4.Data Preprocessing

Before training the model, the dataset was cleaned and prepared using many preprocessing steps to ensure that it is suitable for training a neural network which includes:

- Cleaning column names – Removed extra spaces in column to ensure consistency.
- Target variable conversion – Converted target column (deposit) from categorical values (yes/no) into numerical values (1/0).
- Encoding categorical features – Converted text values columns like (job and education) into numerical form using Label Encoding so that neural network can process them.
- Splitting the dataset – The dataset was divided into training and testing sets. 80% of the data used for training and 20% of the data was used for testing.
- Feature Scaling – Standard Scaler was used to normalise the input features. This step is important because neural networks perform better when input values are on a similar scale.
- Label encoding was used for simplicity, although one-hot encoding may provide better representation for categorical variables

5.Model Design and Implementation

Three models were created using the same structure but with different numbers of hidden layers:

- Model 1 hidden layer: It is the simplest model with only one hidden layer.
- Model 2 hidden layers: It adds an additional hidden layer to capture more complex patterns.
- Model 3 hidden layers: It is a deeper model with three hidden layers.

The following characteristics are shared by all models:

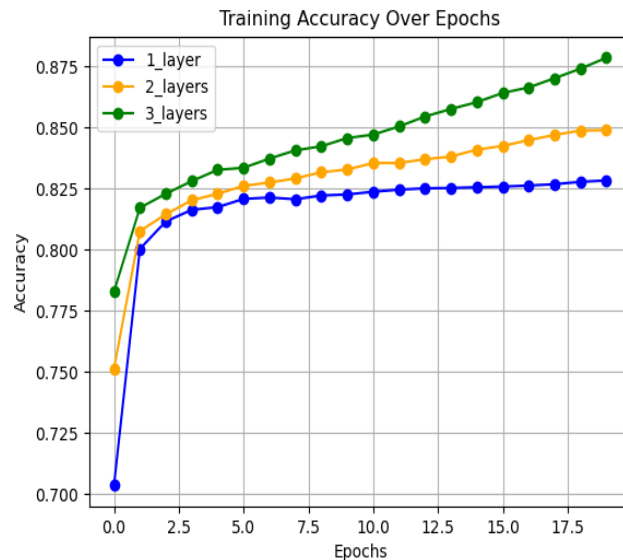
- ReLU activation in hidden layers
- Sigmoid activation in the output layer
- Binary crossentropy loss function
- Adam optimizer

Each model was trained for 20 epochs with a validation split of 0.2. The validation data helps monitor how well the model performs on unseen data during training.

6.Results and Analysis

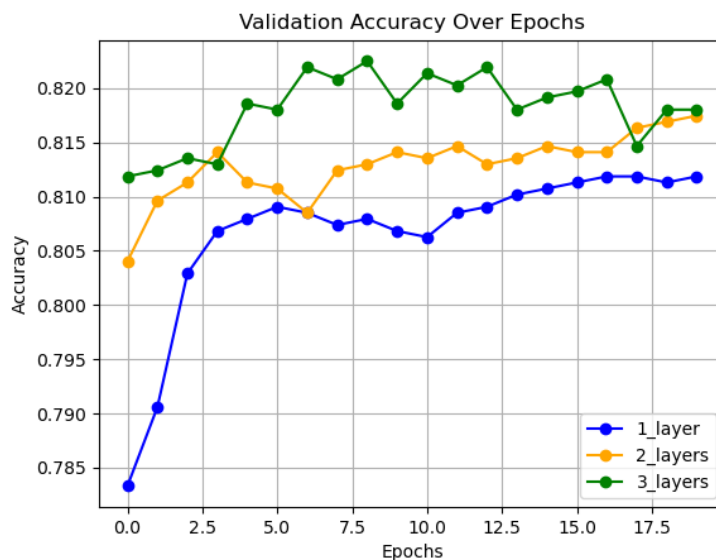
6.1 Training Accuracy vs Epochs

- The training accuracy graph tells how well each model learns from the training data over time. The accuracy of all models improves, when the number of epochs increases indicating that the models are learning patterns from the data.
- The model with two hidden layers performs slightly better than single-layer model.
- The three layer model has high accuracy, but shows insignificant improvement. This shows that adding hidden layers helps improve learning, but the benefit reduces after a certain point.



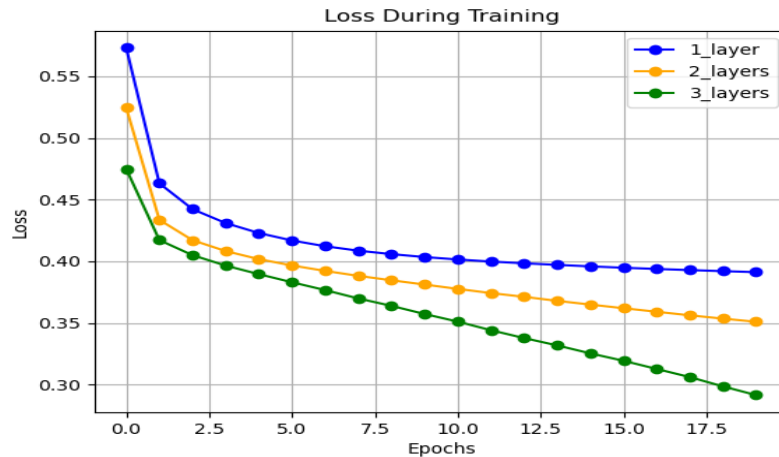
6.2 Validation Accuracy vs Epochs

- The validation accuracy graph tells how well the model performs on unseen data.
- The two-layer model achieves the highest validation accuracy, indicating that it generalises well. The one-layer model performs slightly lower, suggesting it may not capture all patterns in the data.
- The three-layer model shows some fluctuations in validation accuracy, which may indicate overfitting. This means the model is learning the training data too closely and not generalising well.



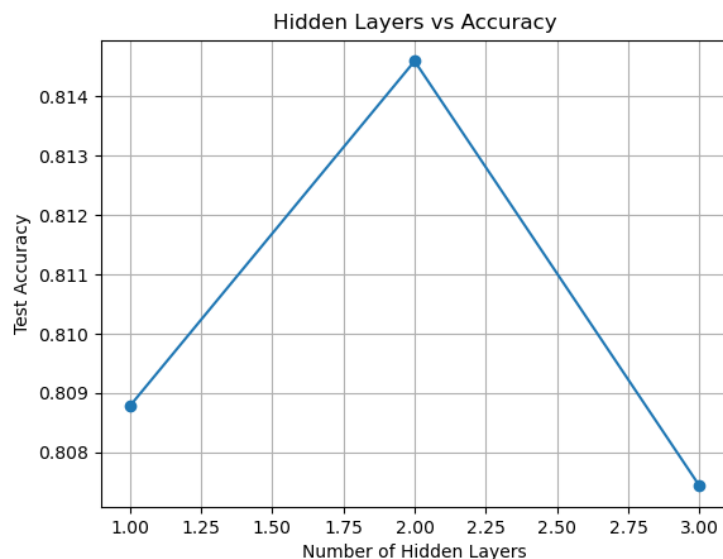
6.3 Loss vs Epochs

- The loss graph shows how the error decreases during training.
- At the start, the loss is high because predictions are not accurate. As training progresses, the loss decreases, showing that the model is learning.
- All models show a decreasing trend in loss, which confirms effective learning. The deeper models reduce loss faster initially, but the improvement slows down later.



6.4 Hidden Layers vs Accuracy

- The graph presents a comparison of the final test accuracies across all models. The single-layer model delivers solid results.
- The two-layer model attains the highest accuracy.
- The three-layer model shows no further improvement and exhibits a slight decline.
- Overall, this indicates that an optimal number of hidden layers exists for this task.



7. Discussion

- The results of this experiment demonstrate that increasing the number of hidden layers improves the model's ability to learn complex patterns. However, adding too many layers may not improve performance.
 - The two-layer model provides a good balance between learning and generalisation. The three-layer model is more complex and may overfit the data, which decreases its performance on unseen data. This shows an important concept in machine learning more complexity does not always lead to better results. It is crucial to choose the right model architecture based on the problem.
- Too few layers → underfitting
Optimal layers → best performance
Too many layers → overfitting
Overfitting is identified through validation performance, as no explicit regularisation techniques were applied.

8. Conclusion

In this assignment, the effect of hidden layers on the performance of a Multilayer Perceptron was analysed using the Bank Marketing Dataset.

The results highlight that:

- Hidden layers enhance the learning ability of the model.
- There is an ideal number of layers for the best performance.
- Too many layers can decrease generalisation and lead to overfitting.
- The two-layer model provides the best balance between learning and generalisation. The three-layer model becomes more complex and does not improve performance. This shows the importance of selecting the right architecture in neural networks
- Future improvements could include regularisation techniques such as dropout or early stopping.

9. Accessibility Considerations

All graphs include clear titles and labelled axes to ensure easy understanding of the results.

Colours are chosen alongside markers and labels to ensure the graphs remain understandable for users with colour vision deficiencies.

Grid lines improve readability and help track trends across epochs. Simple plotting styles with markers are used so that the visualisations are easy to interpret.

10. References

- Moro, S., Cortez, P., & Rita, P. (2014). A Data-Driven Approach to Predict the Success of Bank Telemarketing. *Decision Support Systems*, 62, 22–31.
<https://www.sciencedirect.com/science/article/pii/S0167923614000610>
- UCI Machine Learning Repository. Bank Marketing Dataset.
<https://archive.ics.uci.edu/ml/datasets/bank+marketing>
- Nielsen, M. (2015). *Neural Networks and Deep Learning*.
<http://neuralnetworksanddeeplearning.com/>
- TensorFlow Documentation (Keras API).
<https://www.tensorflow.org/>
- Scikit-learn Documentation.
<https://scikit-learn.org/>